

KAKSHA

Space Debris Tracking & Conjunction-Risk Visualization

कक्षा — Sanskrit for *orbit*



Smart India Hackathon · Problem Statement 83

Technical Explainer for the Team

*Physics calculates. Validation verifies. The risk engine ranks.
Visualization shows. The LLM explains — and nothing else.*

Contents

1. What KAKSHA is, in one minute	2
2. The problem we were given	2
3. The core principle	3
4. Glossary — learn these eight words first	3
5. The pipeline, end to end	4
6. Stage-by-stage walkthrough	4
6.1 Data ingestion — □	4
6.2 TLE parsing — □	5
6.3 Orbital propagation — □	5
6.4 Coordinate frames — □	5
6.5 The simulation clock — □	6
6.6 Screening — □	6
6.7 TCA refinement — □	7
6.8 Encounter geometry and the B-plane — □	7
6.9 Uncertainty — □	7
6.10 Validation — □	8
6.11 Risk ranking — □	9
6.12 The LLM layer — □	9
7. The interface — seven pages	10
The 3D globe	10
Traceability	11
8. Why this is scientifically defensible	11
9. Running it	11
Install	11
Configure	11
Run — two terminals, backend first	12
Verify	12
Ports	12
10. The five-minute demo script	12
11. Questions judges will ask, and the answers	13
12. Who explains what	14

13. Honest limitations 14

14. File map 14

Data sources 15

Space Debris Tracking and Conjunction-Risk Visualization

Smart India Hackathon — Problem Statement

83

Kakshā (कक्षा) is Sanskrit for **orbit**.

1. What KAKSHA is, in one minute

There are roughly 18,700 trackable objects in orbit around Earth. Some are working satellites. Most are junk — dead satellites, spent rocket stages, and fragments from explosions and collisions. They travel at about 7.5 km/s. A 10 cm fragment at that speed carries the kinetic energy of a small car at highway speed.

Satellite operators need to know, in advance, when two of these objects are going to pass close to each other. That event is called a **conjunction**. Knowing about it early means an operator can decide whether to spend fuel on a manoeuvre.

KAKSHA takes public orbital data, propagates every object forward in time, finds every close approach in the next 48 hours, ranks them by risk, shows them on a 3D Earth, and explains each one in plain language.

The important part — the part that makes this a real engineering project rather than a demo — is that **every number is computed by a physics engine, and the AI is only allowed to describe those numbers, never to produce them.**

2. The problem we were given

PS-83: Space Debris Tracking and Conjunction-Risk Visualization

Broken into what it actually demands:

Requirement	Where KAKSHA answers it
Track space debris	Live CelesTrak ingestion, 18,700 objects
Propagate orbits	SGP4/SDP4 engine, verified against reference vectors
Detect conjunctions	Three-stage screening sieve
Assess risk	Weighted, transparent, five-component risk engine
Visualize	Three.js Earth + 2D B-plane encounter view
Explain	LLM layer with mechanical guardrails

3. The core principle

This is the single most important thing to communicate. If a teammate remembers nothing else, remember this:

```
PHYSICS CALCULATES
  ↓
VALIDATION VERIFIES
  ↓
RISK ENGINE RANKS
  ↓
VISUALIZATION SHOWS
  ↓
LLM EXPLAINS
```

The LLM is not the scientist. The numerical engine is the scientist.

The AI layer is structurally forbidden from:

- computing orbital positions
- computing time of closest approach
- computing miss distance or relative velocity
- computing or adjusting uncertainty
- deciding what counts as a conjunction
- changing a risk ranking
- inventing any value

It receives a finished, validated result and writes prose about it. Nothing more. Section 6.11 explains exactly how that is enforced, because "we told it not to" is not enforcement.

4. Glossary — learn these eight words first

Everyone presenting should be fluent in these. Using the wrong word is the fastest way to lose credibility with a technical judge.

Conjunction — a predicted close approach between two orbiting objects. **It is not a collision.** Objects pass within a few kilometres of each other constantly. Say "conjunction" or "close approach", never "collision".

TCA (Time of Closest Approach) — the exact instant two objects are nearest.

Miss distance — the minimum separation between two objects, at TCA.

TLE (Two-Line Element set) — the standard public format for describing an orbit. Two 69-character lines of text. Published by US Space Command and distributed by CelesTrak.

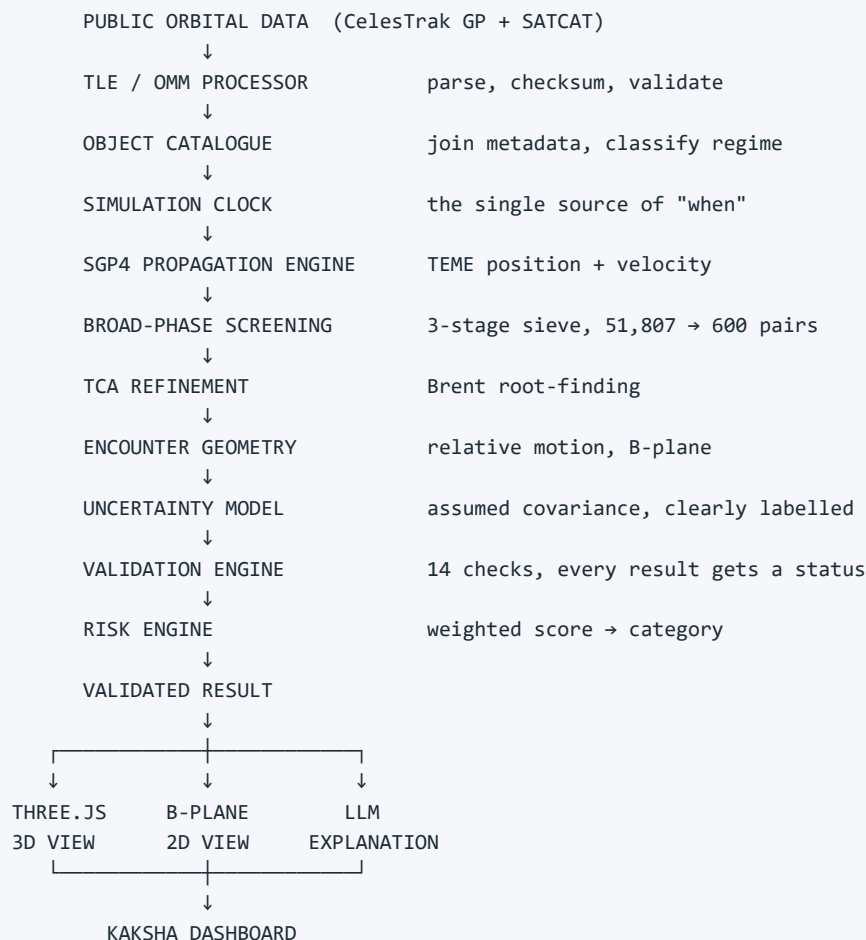
Epoch — the timestamp a TLE describes. Orbital data goes stale; propagating far from the epoch degrades accuracy. Our median element age is under a day.

SGP4 — Simplified General Perturbations 4. The standard analytic model that turns a TLE into a position and velocity at any requested time. TLEs are *defined* by SGP4 — using any other model with a TLE is mathematically wrong.

Covariance — the uncertainty in an object's position, as a 3D error ellipsoid. **Public data does not include it.** This matters enormously; see section 8.

B-plane — the plane perpendicular to the relative velocity vector at TCA. Projecting the encounter onto it turns a 3D near-miss into a readable 2D picture. It is the standard tool in conjunction assessment.

5. The pipeline, end to end



Each arrow is a real module boundary in the codebase. The layers do not reach around each other.

6. Stage-by-stage walkthrough

6.1 Data ingestion — `app/data/providers.py`

We pull from **CelesTrak**, which redistributes the US Space Force public catalogue. Two feeds:

- **GP element sets** — the orbits themselves, fetched by group (active, stations, and specific debris clouds like fengyun-1c-debris, cosmos-2251-debris, iridium-33-debris).
- **SATCAT** — 70,355 rows of metadata: object type, owner country, launch date, decay date, size class.

The provider is an **interface**, not a hardcoded URL. Swapping in Space-Track or a local file means implementing one class. Fetches are cached on disk, and a failed group degrades the run rather than killing it — if one feed 500s, we keep the rest and mark the catalogue degraded: `true`.

Talking point: we don't scrape or fake data. This is the same public catalogue every real SSA tool uses.

6.2 TLE parsing — `app/data/tle_processor.py`

Every line is validated before use:

- **Checksum** — each TLE line carries a modulo-10 checksum digit. We verify it.
- **Field ranges** — eccentricity in $[0, 1)$, inclination in $[0, 180^\circ]$, mean motion positive and physically plausible.
- **Epoch parsing** — the two-digit year rule (57–99 → 1900s, 00–56 → 2000s).
- **Implied decimal points** — TLE stores `0.0001234` as `12340-3`. Getting this wrong silently corrupts the drag term.

Anything malformed is **rejected with a reason**, not silently repaired. The rejection reasons are surfaced on the Validation page.

Current live run: **18,700 parsed, 0 rejected**.

6.3 Orbital propagation — `app/propagation/sgp4_engine.py`

We use the reference `sgp4` library — a direct port of the Vallado/Kelso implementation, with a C accelerator.

Verification: we test against the official Vallado test vector for NORAD 00005 (Vanguard 1). Expected position at epoch:

```
7022.46529266, -1400.08296755, 0.03995155 km
```

Our engine agrees to **7 micrometres**. That is floating-point noise — it is exact agreement. This test runs in CI on every commit.

Two details worth mentioning:

- **WGS-72, not WGS-84.** SGP4 is defined against the older gravity model. Initialising with WGS-84 is a common, subtle error that shifts positions by hundreds of metres.
- **Deep space.** Objects with a period over 225 minutes automatically use the SDP4 extension. We record which model was used per object.

Performance: propagating 18,700 objects across 2,880 timesteps naively would take minutes. We use `SatrecArray`, which evaluates the whole object × time grid inside the C extension. Roughly a hundred times faster.

Failure handling: a propagation that returns a non-finite value, a position inside the Earth, or a speed above 20 km/s is rejected. Failed states become `NaN` rather than plausible-looking garbage — a caller who forgets to check the mask gets an obviously broken number, not a quietly wrong one.

6.4 Coordinate frames — `app/core/frames.py`

This is where naive projects go wrong, and it is worth calling out.

```
TLE → SGP4 → TEME → (rotate by GMST) → ITRF → geodetic lat/lon/alt
```

- **TEME** (True Equator, Mean Equinox) — what SGP4 natively outputs. Inertial: it does not rotate with the Earth.
- **ITRF** — Earth-fixed. What you need for ground tracks and lat/lon.

- **GMST** — Greenwich Mean Sidereal Time, the rotation angle between them.

Every state vector in KAKSHA carries its frame as data, not as a comment. Conjunction analysis happens in TEME (inertial — correct for relative motion). Ground tracks convert to ITRF. The two never mix.

A neat consequence: in the 3D view, satellites are *never* rotated. The Earth mesh spins by GMST instead. That is both physically correct and about twelve thousand times less work than rotating every object every frame.

6.5 The simulation clock — `app/propagation/simulation_clock.py`

One clock drives Earth rotation, Sun direction, the day/night terminator, propagation, conjunction state, and every panel. If it is wrong, everything is wrong *together* rather than subtly inconsistent.

Two modes:

- **REAL_TIME** — simulation time is wall-clock UTC.
- **SIMULATION** — user-chosen anchor plus elapsed time × rate. Scrub forward and back, jump to a TCA, pause, step, run at 1×/10×/100×/1000×.

The clock **refuses** to go more than 30 days from now. SGP4 has no predictive value at that range; plotting it would be theatre.

6.6 Screening — `app/conjunction/screening.py`

This is the algorithmic heart, and the best thing to explain to a judge.

The naive approach: compare every object against every other object at every timestep. For 18,000 objects over 48 hours at 60-second steps that is **4.7×10^{11} pair-tests**. Not a computation — a rumour.

We use a three-stage sieve:

Stage 1 — apogee/perigee filter (no propagation at all). Two objects can only approach if their radial shells overlap:

```
perigee_A - pad ≤ apogee_B   AND   perigee_B - pad ≤ apogee_A
```

Sort by apogee, binary-search the range. $O(n \log n)$ instead of $O(n^2)$. This is the classic first Hoots filter. Pad: 50 km.

Stage 2 — coarse spatial sweep. Propagate survivors onto a 60-second grid in memory-bounded chunks. At each step, a **k-d tree** answers "which objects are near each other" in $O(n \log n)$. We build the tree over the secondaries and query with the primaries, so screening 72 Indian assets against 10,000 objects costs 72 ball-queries per step, not a full neighbour enumeration.

Stage 3 — candidate extraction. Every pair that ever came within the gate gets its coarse minimum recorded and handed to the refiner.

The correctness property that matters most:

A coarse step can *hide* an encounter — two objects closing at 15 km/s travel 900 km between 60-second samples, so the true minimum can sit between two samples that are both far apart. The gate must therefore satisfy:

```
gate ≥ screening_threshold + v_max × Δt / 2
     = 25 km + 16 km/s × 30 s
     = 505 km
```

The screener computes this bound and refuses to run if the configured gate is smaller. It would rather fail loudly than silently miss conjunctions. This is the single most defensible line of code in the project.

Live numbers from a real run:

Catalogue	18,708
After shell filter	3,756
Geometric pairs	51,807
Coarse candidates	16,017
Refined (Brent)	600
Conjunctions	155

6.7 TCA refinement — `app/conjunction/tca.py`

The coarse sweep gives a 60-second bracket. The true closest approach is somewhere inside it, and we need it to sub-second precision.

We do not sample more finely. At TCA, the range-rate is exactly zero:

$$d/dt |r_{rel}|^2 = 2 (r_{rel} \cdot v_{rel}) = 0$$

So finding TCA is a **root-finding problem**, and we solve it with **Brent's method** on the function $g(t) = r_{rel} \cdot v_{rel}$. Brent converges to machine precision in a handful of iterations and cannot overshoot the bracket.

Talking point: "we don't sample for the minimum, we solve for it." A judge who knows numerical methods will notice immediately.

6.8 Encounter geometry and the B-plane — `app/conjunction/bplane.py`

At TCA we compute relative position, relative velocity, relative speed, and the encounter angle.

Then we build the **B-plane** — the plane perpendicular to relative velocity:

- $\hat{\eta} = v_{rel} / |v_{rel}|$ — normal to the plane
- $\hat{\xi} = (v_2 \times v_1) / |v_2 \times v_1|$ — lies in the plane
- $\hat{\zeta} = \hat{\xi} \times \hat{\eta}$ — completes the right-handed triad

At TCA, $r_{rel} \cdot v_{rel} = 0$ by definition, so the relative position vector already lies in the B-plane. Its components (b_{ξ}, b_{ζ}) are the miss vector.

This is the standard Foster formulation used by NASA CARA. The 2D view in the UI is the real projection, plotted from these numbers — not an illustration.

6.9 Uncertainty — `app/uncertainty/models.py`

This is the section that demonstrates scientific integrity, and it is worth being loud about.

Public TLE data **does not publish covariance**. Real operational conjunction assessment uses covariance from the tracking network, which is not public.

We therefore apply a **documented, assumed** error model in the RIC frame (radial / in-track / cross-track), 1-sigma at epoch:

Axis	σ at epoch	Growth per day
Radial	0.20 km	0.15 km/day
In-track	1.00 km	1.20 km/day
Cross-track	0.40 km	0.30 km/day

In-track error dominates and grows fastest — that is physically correct, since along-track position is the most sensitive to drag mismodelling.

Both objects' covariances are combined, projected into the B-plane, and used to compute a Mahalanobis distance and a miss-over-sigma ratio.

We compute a probability figure, and we label it honestly. Because the covariance is assumed rather than measured, the field is named `conditional_encounter_probability`, `covariance_source` reads `ASSUMED_MODEL`, and `is_operational_pc` is false. The UI states, in words, that this is conditional on an assumed model and is not an operational probability of collision.

We never call a heuristic "probability of collision". Doing so is the single most common way these projects lose scientific credibility.

Hard-body radius defaults to 5 m when object dimensions are unknown, and the source of that value is reported alongside it.

6.10 Validation — `app/validation/engine.py`

Nothing reaches the dashboard unvalidated. **21 named checks** — 14 on every conjunction, 7 on the catalogue as a whole.

Per conjunction (14):

Check	What it catches
<code>miss_distance_reproducible</code>	Recomputing the separation disagrees with the stored value
<code>tca_is_stationary_point</code>	Range-rate at the reported TCA isn't actually zero
<code>tca_solver_converged</code>	Brent failed to converge
<code>tca_within_screening_window</code>	TCA landed outside the requested window
<code>miss_vector_lies_in_bplane</code>	Miss vector has an out-of-plane component it shouldn't
<code>bplane_basis_orthonormal</code>	The B-plane triad isn't orthonormal
<code>bplane_miss_matches_3d</code>	2D projection disagrees with the 3D separation
<code>separation_within_screening_volume</code>	Reported event exceeds the screening threshold
<code>distinct_objects</code>	An object paired with itself
<code>element_sets_fresh</code>	Elements older than 14 days (warning at 7)
<code>states_share_an_epoch</code>	The two states were evaluated at different times
<code>metadata_available</code>	Missing country/operator attribution
<code>covariance_published</code>	Records that covariance is assumed, not measured

Check	What it catches
linear_encounter_assumption	Encounter too slow/curved for the linear B-plane model

Per catalogue (7): catalog_loaded, feed_reachable, feed_note, median_element_age, stale_object_fraction, records_rejected_at_parse, attribution_complete.

Every conjunction carries a status: **VALIDATED**, **WARNING**, **INVALID**, or **INSUFFICIENT_DATA**. Invalid results are not displayed — and the explanation layer **refuses to explain them**, on the grounds that explaining an invalid result lends it credibility it has not earned.

6.11 Risk ranking — app/risk/engine.py

A transparent weighted score, 0–100:

$$\text{score} = 100 \times \sum (\text{weight}_i \times \text{normalised}_i)$$

Component	Weight	Rationale
Miss distance	0.40	Closest is most dangerous
Uncertainty ratio	0.30	Miss ÷ combined sigma
Relative velocity	0.15	Energy of a potential impact
Time to TCA	0.10	Sooner = less reaction time
Object class	0.05	Debris vs active vs station

Category boundaries:

Category	Score
CRITICAL	≥ 75
HIGH	≥ 55
MODERATE	≥ 30
LOW	< 30

Every component's raw value, normalised value, weight and point contribution is returned with the result. The UI can therefore answer "why is this ranked

#1?" by pointing at actual arithmetic — the WHY tab shows the full breakdown, and the numbers add up to the displayed score.

The formula lives in exactly one place. The frontend never computes a risk label; it renders one.

6.12 The LLM layer — app/llm/

Input: a validated result. Output: prose. That is the entire contract.

Enforcement is in three independent layers, because a system prompt alone is a request, not a control:

1. Structural. The prompt contains only finished numbers. There is no tool, no catalogue access, no propagator in scope. The model *cannot* compute an orbit because it has nothing to compute with. Automatic function calling

is explicitly disabled in the SDK config — no tools are declared, so this changes nothing today; it makes it impossible for a later edit to change it by accident.

2. Instructional. A system prompt stating the rules: never perform orbital mechanics, never invent a value, never change a risk category, never say "collision", never claim an operational probability of collision, say "data unavailable" rather than filling a gap.

3. Mechanical — this is the one to demo. After generation, we **extract every numeral from the output and trace it back to a supplied value**. Anything unverified is flagged. We also scan for overstated claims. The audit result is returned to the client and displayed as a badge. If the model invents a number, the UI says so.

Provider-agnostic. The layer runs on either Anthropic or Google Gemini behind one interface. Provider is auto-detected from the model id first, then the key format, and can be forced with KAKSHA_LLM_PROVIDER. Currently configured for gemini-2.5-flash.

Graceful degradation. With no key, a failed call, a rate limit or a network error, the panel falls back to a **deterministic template** built directly from the numbers and clearly labelled as such. The explanation panel is never empty and never fabricated.

7. The interface — seven pages

Page	What it shows
Dashboard	3D Earth, ranked conjunction rail, analysis panel, live counters
Tracker	Catalogue browser, search, filters, per-object detail
Conjunctions	Full screening results, sortable, filterable by risk
Calculations	The same event traced through all nine pipeline stages
Analysis	Distributions — risk, miss distance, relative velocity, country
Simulation	Time controls, jump-to-TCA, playback
Validation	Data quality, propagation status, checks, model limitations

The 3D globe

- Photoreal Earth: NASA albedo, city-lights night texture, cloud layer, normal-mapped relief, atmospheric limb scattering.
- **Real terminator** — computed from Vallado's analytic solar ephemeris, not a fudged offset. The day/night line is where it actually is right now.
- **Real rotation** — the mesh spins by GMST.
- **Per-class sprites** — satellites, stations, rocket bodies and debris each have a distinct silhouette, drawn procedurally to an offscreen canvas and used as point sprites. Shape carries object class; colour carries ownership and risk. Four draw calls for the entire catalogue.
- **Display sampling** — the globe draws a readable subset; screening always runs against the full catalogue. Drawing 18,700 points at once is legible as a density field but useless for picking out an asset.

Traceability

Every displayed conjunction is traceable end to end: click an event → see its two objects → their element sets and epochs → propagated states → TCA → miss distance → relative velocity → uncertainty → validation status → risk breakdown → LLM explanation. The **Calculations** page walks exactly that chain.

8. Why this is scientifically defensible

Five claims we can back up under questioning:

1. The propagation is the reference implementation, and we prove it. Agreement with the published Vallado vector to 7 micrometres, tested in CI.

2. TCA is solved, not sampled. Brent root-finding on the range-rate.

3. The screening gate is proven safe, not assumed. The code derives the minimum safe gate from the step size and worst-case closing speed, and refuses to run below it.

4. We do not overclaim on probability. No covariance in public data → no operational Pc. We say so in the field names, the API, and the UI.

5. Real-time calculation, not real-time measurement. We propagate published elements; we do not observe satellites. The UI always shows element epoch, data age, propagation time and simulation time. Claiming live tracking would be false, and we don't.

175 automated tests cover TLE parsing, SGP4, frame transforms, relative motion, TCA, B-plane geometry, uncertainty, risk scoring, LLM guardrails, provider routing, and full-pipeline integration. They run offline against fixed reference vectors, so a green suite means the numerics are sound even with no network.

9. Running it

Prerequisites: Python 3.11+, Node 18+, internet on first run.

Install

```
cd backend && python -m venv .venv && .venv/Scripts/python -m pip install -r requirements.txt
```

```
cd frontend && npm install
```

Configure

backend/.env is not in the repository — it holds secrets. Create your own:

```
cp backend/.env.example backend/.env
```

Everything in it is optional. For the generated explanation layer, add one key:

```
KAKSHA_GEMINI_API_KEY=...
KAKSHA_LLM_MODEL=gemini-2.5-flash
```

or

```
KAKSHA_ANTHROPIC_API_KEY=sk-ant-...
KAKSHA_LLM_MODEL=claude-opus-5
```

Without a key everything still works; the explanation panel serves the deterministic template.

Run — two terminals, backend first

```
cd backend && .venv/Scripts/python -m uvicorn app.main:app --port 8000
```

```
cd frontend && npm run dev
```

Open **http://localhost:5173**.

On Windows, `start-backend.cmd` and `start-frontend.cmd` do the same and create the `venv` or install `node_modules` if missing. In PowerShell prefix with `.\` and quote paths containing spaces.

First launch fetches ~18,700 element sets plus SATCAT and takes 30–60 seconds. Later starts use the cache.

Verify

```
cd backend && .venv/Scripts/python -m pytest -q
```

175 tests, about 16 seconds, fully offline.

Ports

Backend CORS allows `localhost:5173` and `127.0.0.1:5173` only. If Vite starts elsewhere because 5173 is taken, the browser blocks every API call. Free 5173 rather than letting Vite pick another port.

10. The five-minute demo script

0:00 — Frame the problem. "18,700 tracked objects, most of them junk, moving at 7.5 km/s. Operators need to know about close approaches before they happen."

0:30 — The dashboard. Point at the globe. Real terminator, real rotation, live catalogue. Note the bottom strip: total objects, debris count, risk counters — all live.

1:15 — The pipeline badge. Point at the screening panel on the globe: 18,708 → 3,756 → 51,807 → 16,017 → 600 → 155. "That's the sieve doing its job. We never compute 4.7×10^{11} pair tests."

1:45 — Click conjunction #1. Right panel opens. Names, miss distance, TCA, relative velocity, risk category. Globe highlights both objects and their orbits.

2:15 — The B-PLANE tab. "This is the encounter projected onto the plane perpendicular to relative velocity. Standard conjunction-assessment tool. The ellipse is the combined position uncertainty."

2:45 — The WHY tab. "Here's why it's ranked first — five weighted components, raw values, contributions. The arithmetic is visible and it adds up."

3:15 — The EXPLAIN tab. Generated explanation appears. **Then point at the audit badge.** "Every number in that text was traced back to a value the physics engine produced. If the model invented one, this badge would say so."

3:45 — CALCULATIONS page. "Same event, nine stages, raw TLE through to risk score. Full traceability."

4:15 — VALIDATION page. "And this is what the system admits it can't do. Public data has no covariance, so we use a documented assumed model and we never call our output a probability of collision."

4:45 — Close on the principle. "Physics calculates. Validation verifies. The risk engine ranks. The LLM only explains — and we check its arithmetic."

11. Questions judges will ask, and the answers

"Is this real data or simulated?" Real. CelesTrak's public GP catalogue plus SATCAT metadata — the same source professional tools use. 18,700 objects, median element age under a day.

"Are you tracking these live?" No, and we're careful about that wording. We *propagate* published orbital elements. Nobody outside a tracking network measures positions live. The UI always shows element epoch and data age, and we call it real-time *calculation*, not real-time measurement.

"Is the AI doing the orbital mechanics?" No. The AI cannot compute anything — it has no tools and no data beyond the finished numbers. And we verify: every numeral in its output is traced back to a physics-engine value, with the audit shown in the UI.

"How do you know your propagation is correct?" We test against the official Vallado SGP4 verification vector and agree to 7 micrometres. That test runs on every commit.

"How do you handle 18,000 objects without it taking forever?" Three-stage sieve — apogee/perigee shell filter, then k-d tree spatial queries, then Brent refinement only on survivors. 51,807 geometric pairs collapse to 155 real conjunctions in about seven seconds.

"What's your probability of collision?" We deliberately don't report one. Public data has no covariance, and Pc without covariance is meaningless. We report a conditional encounter probability under a documented assumed error model, clearly labelled, plus a transparent risk score. Overclaiming here would be the easiest way to be wrong.

"Could you miss a conjunction?" The coarse step could hide one if the gate were too small — so the code derives the minimum safe gate from step size and worst-case closing speed (505 km at our settings) and refuses to run below it.

"What happens if CelesTrak is down?" Cached data is used and the catalogue is marked degraded, with data age shown. If a single feed fails, the others still load. Nothing silently pretends to be fresh.

"Why is it called KAKSHA?" कक्षा — Sanskrit for orbit.

12. Who explains what

Suggested split so nobody has to know everything:

Person 1 — Problem and product. Sections 1, 2, 7. The debris problem, what the product does, live walkthrough of the dashboard and the seven pages.

Person 2 — Physics and numerics. Sections 6.3–6.8. SGP4, the Vallado verification, coordinate frames, the screening sieve, Brent root-finding, the B-plane.

Person 3 — Risk, uncertainty and validation. Sections 6.9–6.11, and section 8. The assumed covariance model, why we refuse to say "probability of collision", the validation engine, the risk formula.

Person 4 — AI layer and architecture. Section 3, 6.12, and the file map. The containment argument, the three enforcement layers, the numeric audit, provider-agnostic design.

Everyone should know the **glossary (section 4)** and the **core principle (section 3)**.

13. Honest limitations

Stating these *before* a judge finds them is a strength, not a weakness.

- **No real covariance.** Assumed model only. Everything downstream of it is conditional, and labelled as such.
 - **SGP4 is a short-arc theory.** Accuracy degrades days from epoch. The clock refuses to propagate beyond 30 days.
 - **Screening subset.** The default screen caps at 4,000 secondaries per run for interactive latency. The architecture supports the full catalogue; it is a latency choice, not a limit.
 - **No manoeuvre modelling.** Active satellites manoeuvre; public elements do not announce it. A post-manoeuve conjunction prediction is stale.
 - **No atmospheric density forecasting.** Drag uses the TLE's B^* term. Real operations use space-weather-driven density models.
 - **Not for operational collision avoidance.** This is a screening and visualization system. Real avoidance decisions need operator covariance, tracking updates and owner coordination.
-

14. File map

backend/app/	
core/	
config.py	all tunable constants, one place
frames.py	TEME ↔ ITRF ↔ geodetic, the only conversions
timebase.py	UTC, Julian dates, GMST, solar ephemeris
logging.py	structured JSON logging, per-stage
data/	
providers.py	CelesTrak client, cache, swappable interface
tle_processor.py	parsing, checksums, validation
metadata.py	SATCAT join, object types, orbital regimes
catalog.py	the fused catalogue, filters, search
propagation/	
sgp4_engine.py	SGP4/SDP4, scalar + vectorised
simulation_clock.py	the single source of "when"
conjunction/	
screening.py	three-stage sieve
tca.py	Brent refinement
encounter.py	relative motion, event assembly
bplane.py	Foster B-plane geometry
uncertainty/	
models.py	assumed covariance, projection, Mahalanobis
validation/	
engine.py	14 checks, four statuses
risk/	
engine.py	weighted score, categories, explainability
llm/	
providers.py	Anthropic + Gemini behind one interface
explainer.py	prompt construction, deterministic fallback
guardrails.py	numeric audit, claim scanning
api/routes/	catalog, propagation, conjunctions, analysis
services/	screening orchestration and caching
schemas/	response serialisation
frontend/src/	
components/globe/	Earth, sprites, object field, orbits
components/bplane/	2D encounter view
components/panels/	left rail, right rail, hover card, explanation
components/layout/	top bar, stat strip, boot gate
components/charts/	analysis distributions
pages/	the seven routes
api/	typed client
store/	clock mirror, selection, filters
hooks/	data fetching

Data sources

- **CelesTrak** — GP element sets and SATCAT metadata (celestrak.org). Redistribution of the US Space Force public catalogue.
- **NASA Visible Earth** — Blue Marble albedo, city lights, cloud textures.

KAKSHA — Smart India Hackathon PS-83. Physics calculates. Validation verifies. The risk engine ranks. Visualization shows. The LLM explains — and nothing else.